

Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks

[논문 발제]

24기 분석 조희선, 서혜현

CONTENTS

INTRODUCTION + Background

Related Work

Faster R-CNN

EXPERIMENTS

CONCLUSION

1. Introduction

- 기존 객체 탐지 기술의 발전은 주로 Region Proposal 방법과 Region-based CNN(R-CNN)에 의해 이루어졌음.
- Fast R-CNN이 제안되어 속도는 빨라졌지만, Region Proposal(영역 제안) 단계가 병목이었음.
- Selective Search 같은 기존 방법은 CPU 기반으로 느리고 GPU의 장점을 살리지 못함.
- 본 논문은 이 병목을 해결하기 위해 Region Proposal Network(RPN)을 도입하여 객체 탐지 전체를 거의 실시간으로 수행할 수 있도록 함.
- RPN과 Fast R-CNN을 통합해 하나의 네트워크로 만들었고, 이를 Faster R-CNN이라 명명.

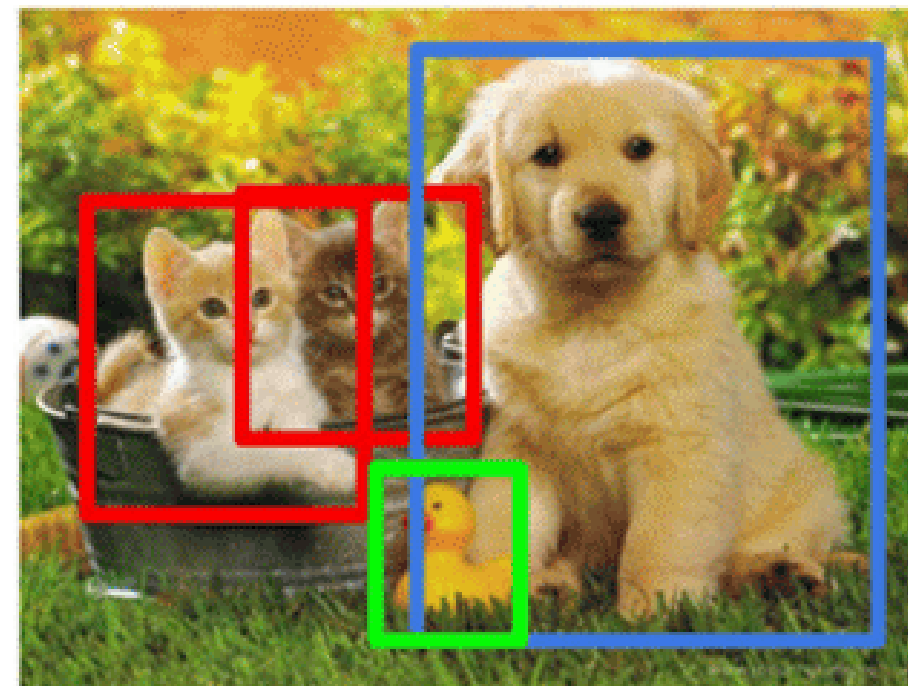
Object Detection

Classification



CAT

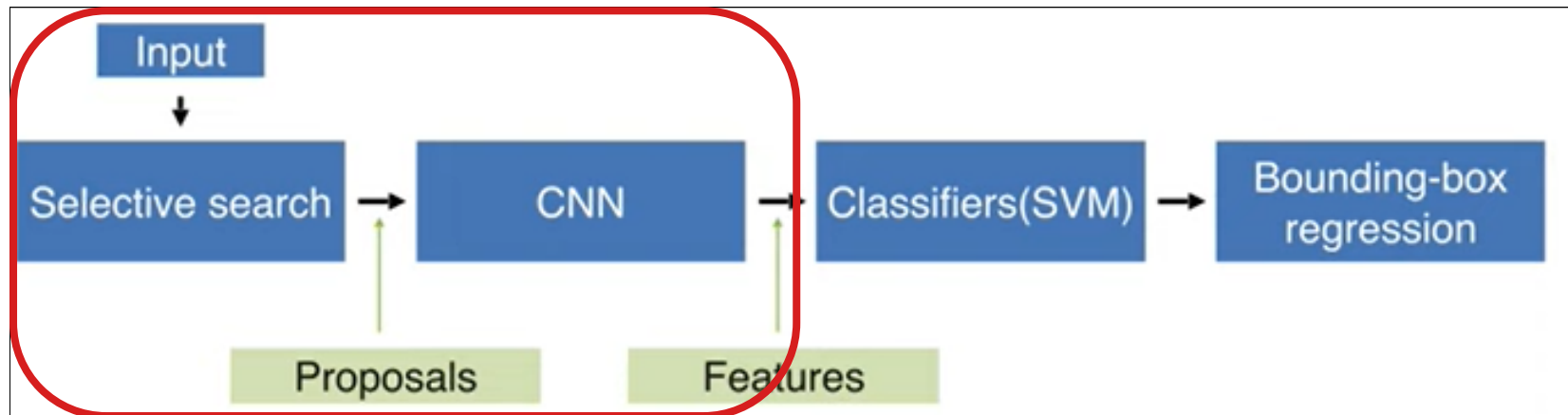
Object Detection



CAT, DOG, DUCK

01 Background

R-CNN



1. 입력 (Input)

- 원본 이미지를 입력으로 받는다.

2. Region Proposal (영역 후보 생성)

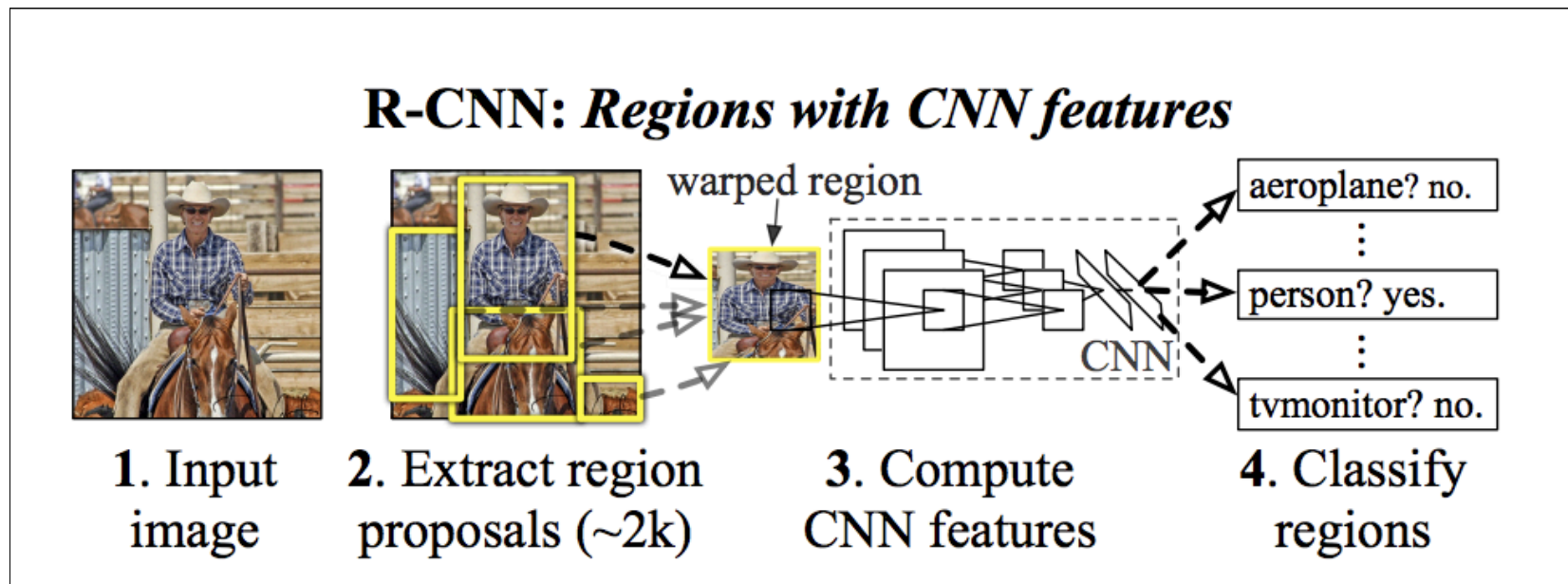
- Selective Search 알고리즘을 사용해서 이미지 안에서 객체가 있을 법한 영역(Region Proposals)을 생성한다.
- 보통 2000개 후보 영역 만듦.

3. Warping (크기 조정)

- 각 후보 영역(Proposal)을 CNN이 처리할 수 있도록 고정된 크기로 크기 조정(Warping) 한다.

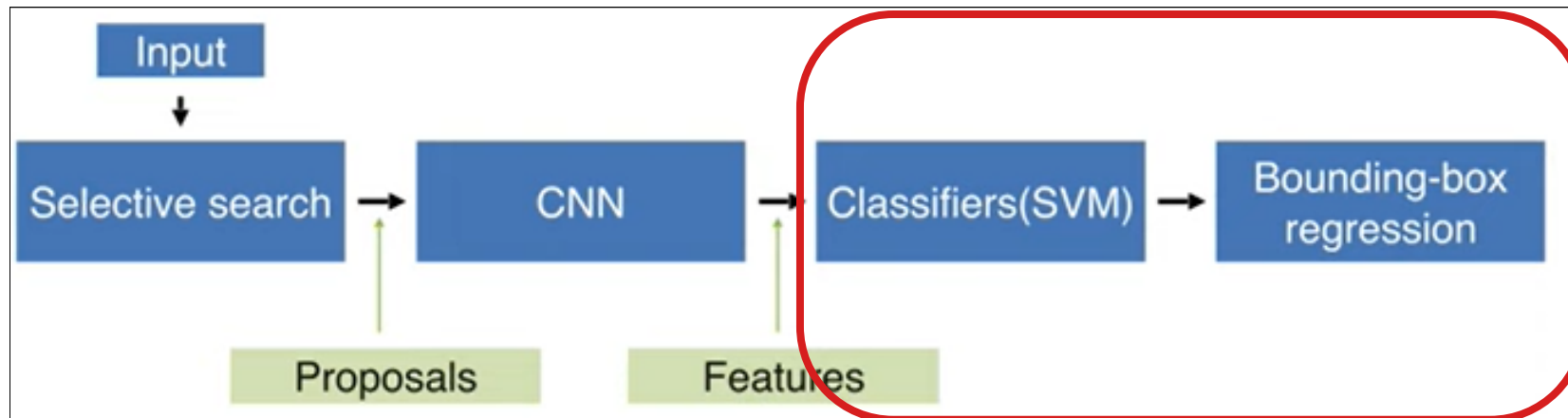
4. Feature Extraction (특징 추출)

- 크기 조정된 각 후보 영역을 CNN에 넣어서 특징 벡터(feature vector)를 추출한다.



01 Background

R-CNN

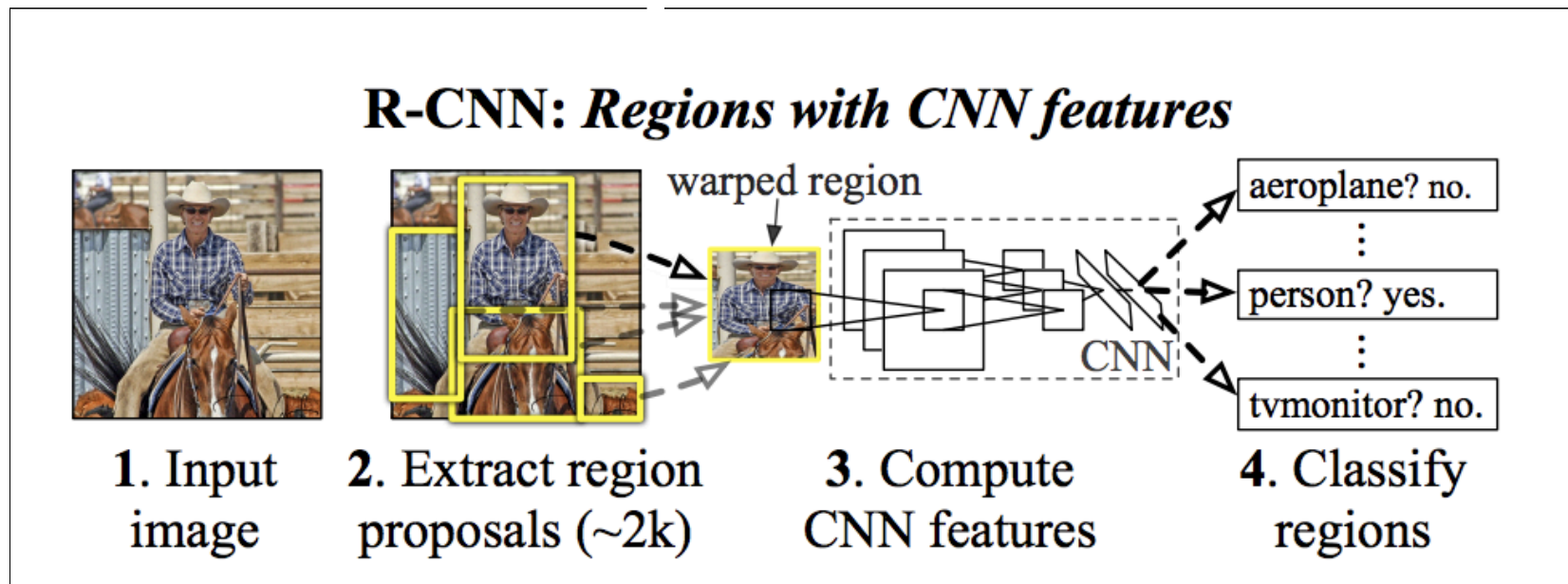


5. Classification (객체 분류)

- 추출한 특징을 이용해서 각 영역이 어떤 객체인지 SVM으로 분류한다.
 - 객체 클래스가 뭔지 예측하는 단계.
 - 여기서 배경(Background)인지 아닌지도 분류함.

6. Bounding Box Regression (정교한 위치 조정)

- 분류된 객체의 경계 상자(Bounding Box)가 더 정확하게 위치하도록 조정한다.
- 객체가 어디에 정확하게 있는지 좌표를 다듬어 주는 역할



01 Background

R-CNN

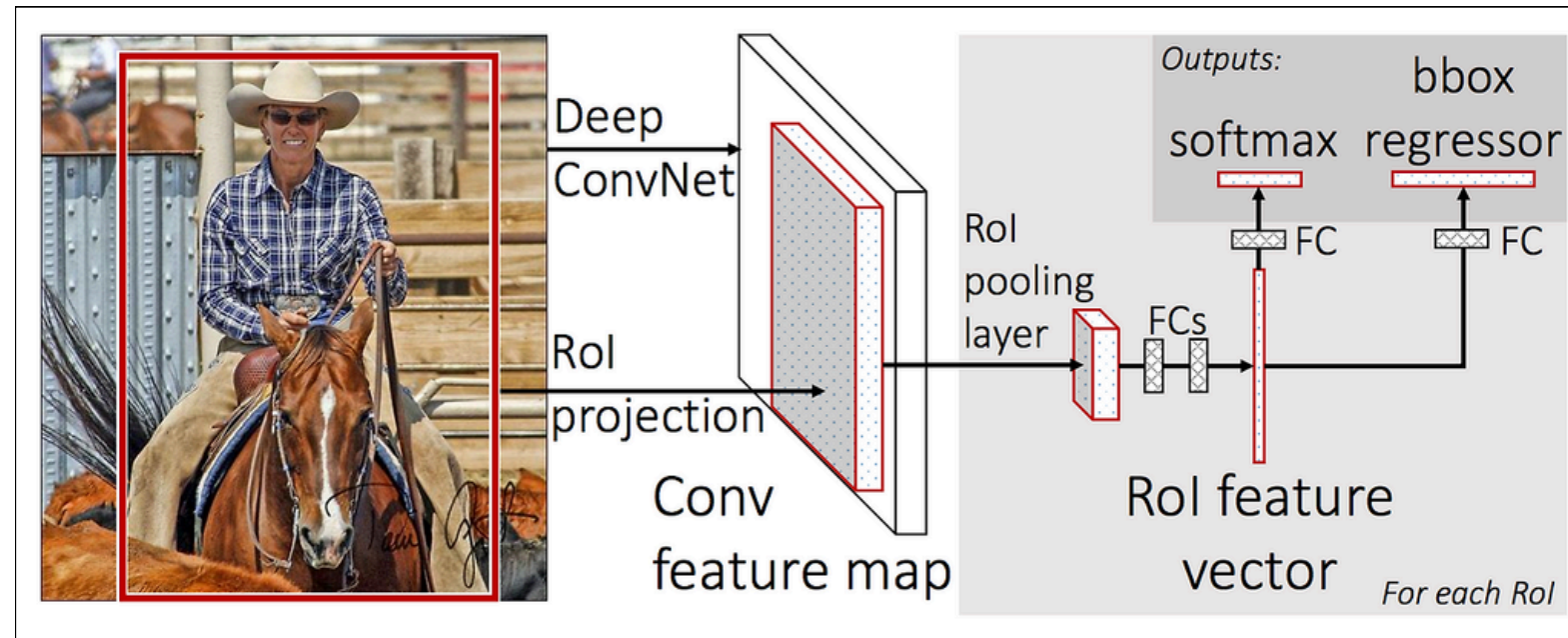
한계점

- 합성곱 신경망의 입력을 위한 고정된 크기를 위해 warping/ crop 을 사용해야 하며 그 과정에서 이미지 정보 손실이 일어난다.
- 2000여개의 영역마다 CNN을 적용해야 하기에 학습 시간이 오래 걸린다.
- 학습이 여러 단계로 이루어지며 이로 인해 긴 학습 시간과 대용량 저장 공간이 요구된다.
- Object Detection의 속도 자체도 느리다.

01 Background

Fast R-CNN

R-CNN의 비효율적인 후보 영역 처리 문제를 해결하기 위해 등장



1. 입력 (Input)

2. Feature Extraction (특징 추출)

- CNN을 한 번만 실행하여 전체 이미지의 Feature Map(특징 맵)을 생성한다.
→ R-CNN과 다르게 후보 영역별 CNN을 돌리는 것이 아니라, 먼저 전체 Feature Map을 구함!

3. Region Proposal (영역 후보 생성)

- Selective Search 알고리즘을 사용하여 객체가 있을 법한 영역을 생성한다.
- 하지만 CNN을 먼저 실행했기 때문에 후보 영역을 CNN에 각각 넣지 않아도 됨
→ 속도 개선!

4. ROI Pooling (Region of Interest Pooling)

- Feature Map에서 후보 영역에 해당하는 부분만 추출하여 고정된 크기로 변환한다.

5. Classification (객체 분류)

- Fully Connected Layer + Softmax 분류기를 사용하여 객체 클래스를 예측한다.
- 배경(Background)인지 아닌지도 분류함.

6. Bounding Box Regression (정교한 위치 조정)

- 분류된 객체의 Bounding Box(경계 상자)를 더 정확한 위치로 조정

01 Background

Fast R-CNN

Fast R-CNN에서 특징맵을 뽑으면 효율적인 이유?

Fast R-CNN은 R-CNN과 다르게 원본 이미지를 입력 받아서 CNN을 먼저 이용해서 특징맵을 추출함

R-CNN 방식

1. **Selective Search**를 이용해 후보 영역을 2000개 뽑음.
2. 각 후보 영역을 CNN에 넣어서 특징을 추출 → CNN을 2000번 실행해야 함 → 속도 느림

Fast R-CNN / Faster R-CNN 방식

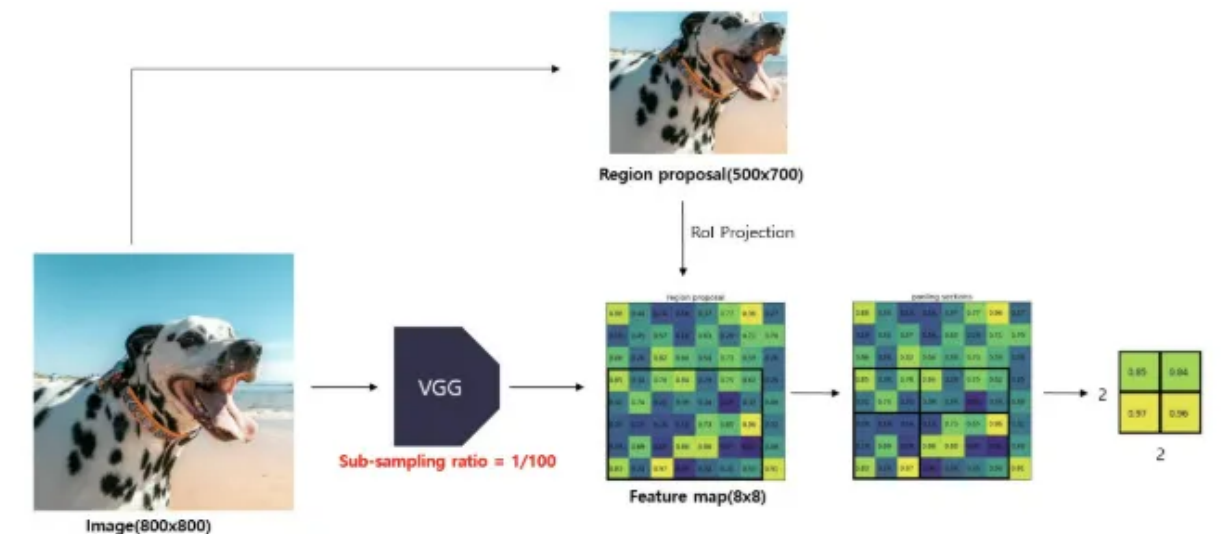
1. 전체 이미지에 대해 CNN을 먼저 한 번만 실행해서 특징맵을 만들고,
2. 그 특징맵에서 후보 영역을 추출 (ROI Pooling 사용)
3. 이후 후보 영역을 분류하고 바운딩 박스를 조정.

01 Background

Roi Pooling

Roi Pooling(Region of Interest Pooling)은 CNN에서 생성된 Feature Map에서 후보 영역(Region of Interest, Roi)에 해당하는 부분만 추출하고, 고정된 크기로 변환하는 연산

→ 크기가 제각각인 후보 영역들을 동일한 크기로 변환하는 과정



[Roi Pooling이 중요한 이유]

1. CNN이 먼저 Feature Map을 생성하므로, CNN을 한 번만 실행해도 여러 개의 후보 영역을 처리 가능!

- R-CNN처럼 각 후보 영역을 CNN에 개별적으로 넣는 방식보다 훨씬 빠름 → 연산량 감소

2. 고정된 크기의 입력을 Fully Connected Layer(FC Layer)에 전달할 수 있음!

- FC Layer는 고정된 크기의 입력을 필요로 하기 때문에,
- ROI Pooling을 통해 후보 영역을 동일한 크기로 변환해야 함.

01 Background

Fast R-CNN

Multi Task 진행

Fast R-CNN은 R-CNN과 달리 Multi Task를 진행함.

- R-CNN → 각 영역에 대해 객체 분류를 별도로 하고, 해당 객체의 위치를 후처리로 예측함.
→ 객체 분류와 위치 예측이 따로 이루어짐.
- Fast R-CNN → 전체 이미지를 한 번에 CNN에 통과시킨 후, ROI pooling을 사용하여 관심 영역을 추출
→ 추출된 특징에 대해 객체 분류와 위치 예측을 동시에 수행

01 Background

Fast R-CNN

Fast R-CNN 한계

- **Testing 단계에서 여전히 느리다.**
- Why? Region proposal 단계가 대부분의 시간을 차지하기 때문.
- R-CNN과 Fast R-CNN은 Selective Search(Region Proposal)에서 CPU이용 → 속도 느림



Faster R-CNN 도입 : Region proposal에 신경망을 적용하여 시간을 단축하자!
(Selective Search에서 GPU사용하겠다!) = RPN(Region Proposal Network 도입)

2. Related Work

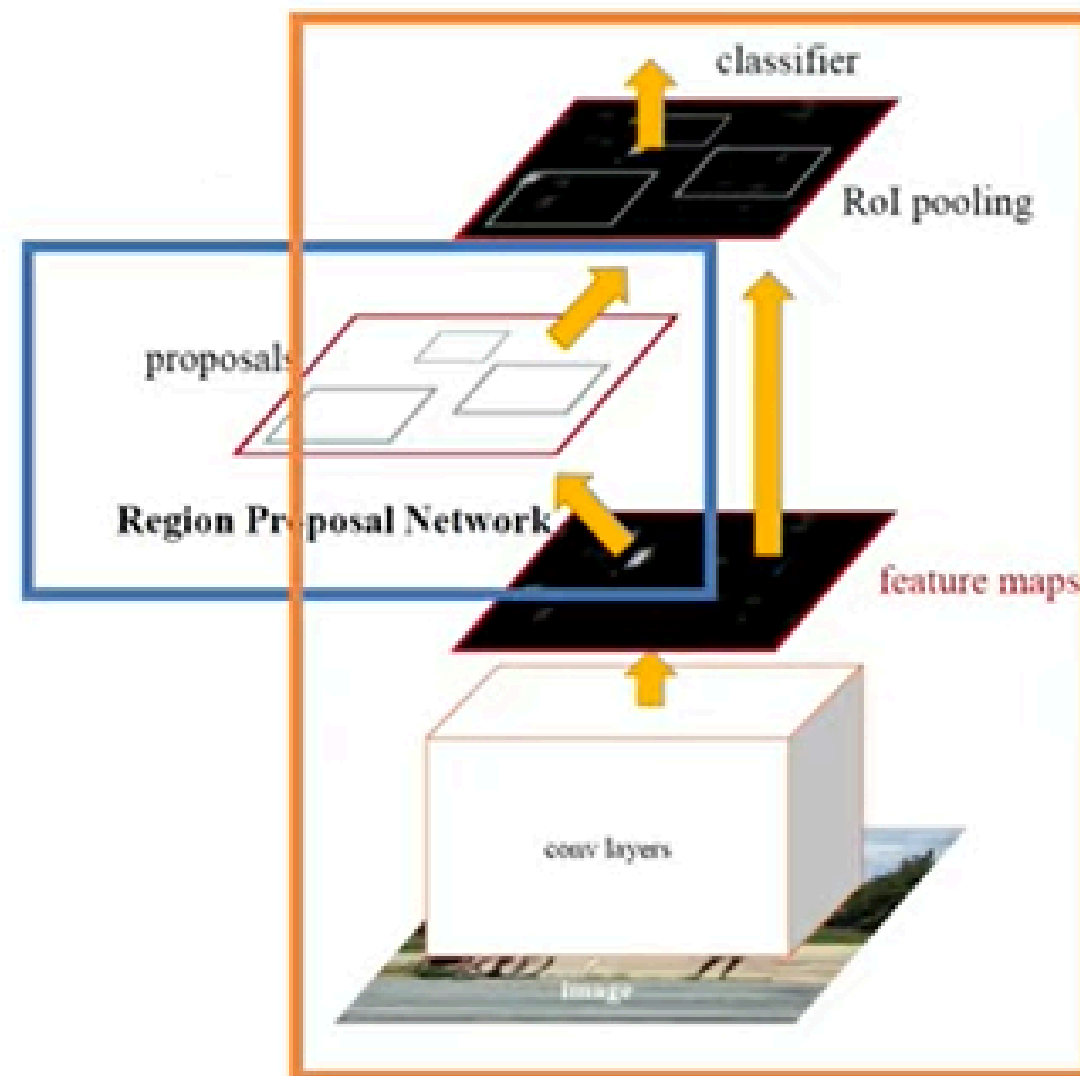
Object Proposals

1. Region Proposal 을 생성하는 방법 연구가 활발함
2. 대표적인 방법:
 - 초픽셀 기반: Selective Search, CPMC, MCG
 - 슬라이딩 윈도우 기반: Objectness in Windows, EdgeBoxes
3. 기존 객체 탐지 모델(R-CNN, Fast R-CNN)은 객체 제안과 탐지를 독립적으로 수행.

Deep Networks for Object Detection

- R-CNN: CNN을 활용하여 제안된 영역을 분류(객체 or 배경). 하지만 바운딩 박스를 직접 예측하지 않음.
객체 바운딩 박스 예측 방법
- OverFeat: 완전연결층을 학습하여 객체 위치를 예측, 이후 컨볼루션 층으로 변환하여 다중 객체 탐지 수행.
- MultiBox: 여러 개의 클래스 무관 바운딩 박스를 동시에 예측, 하지만 특징을 공유하지 않음.

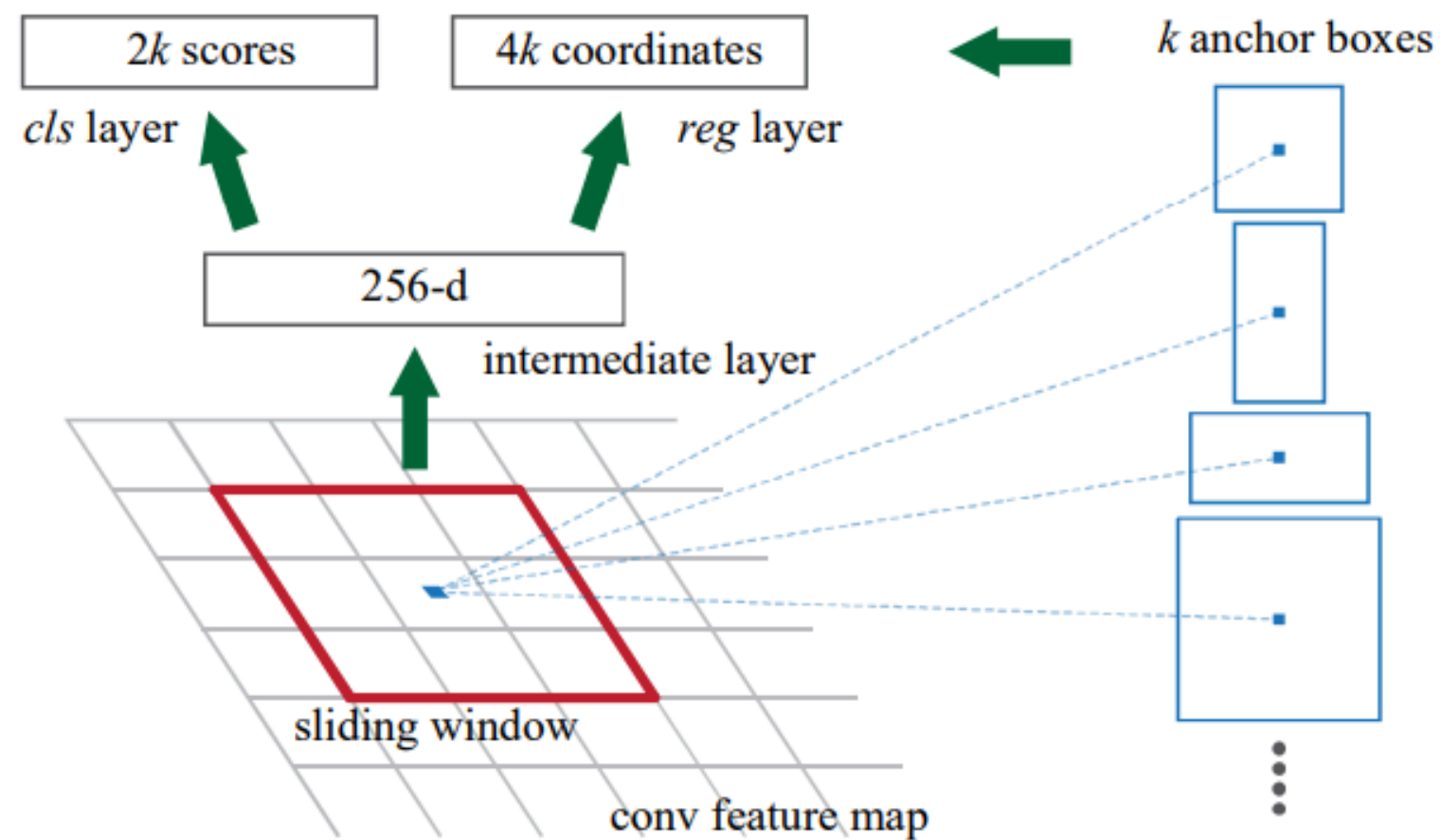
3. Faster R-CNN



$$\boxed{\text{RPN}} + \boxed{\text{Fast RCNN}}$$

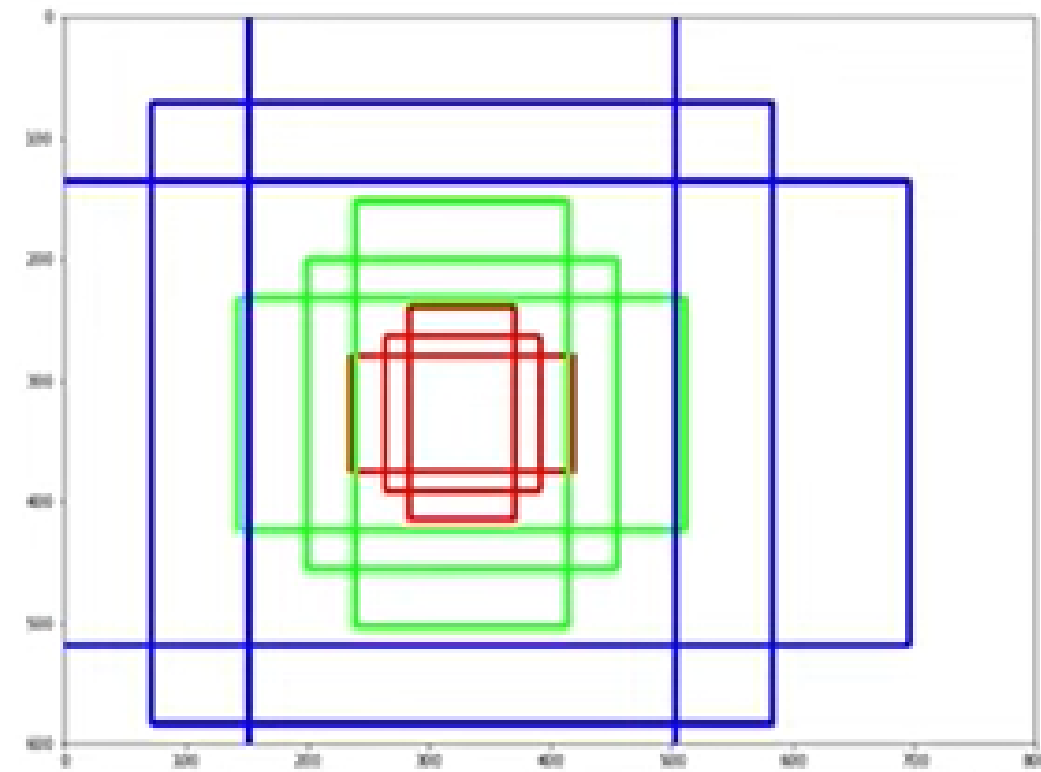
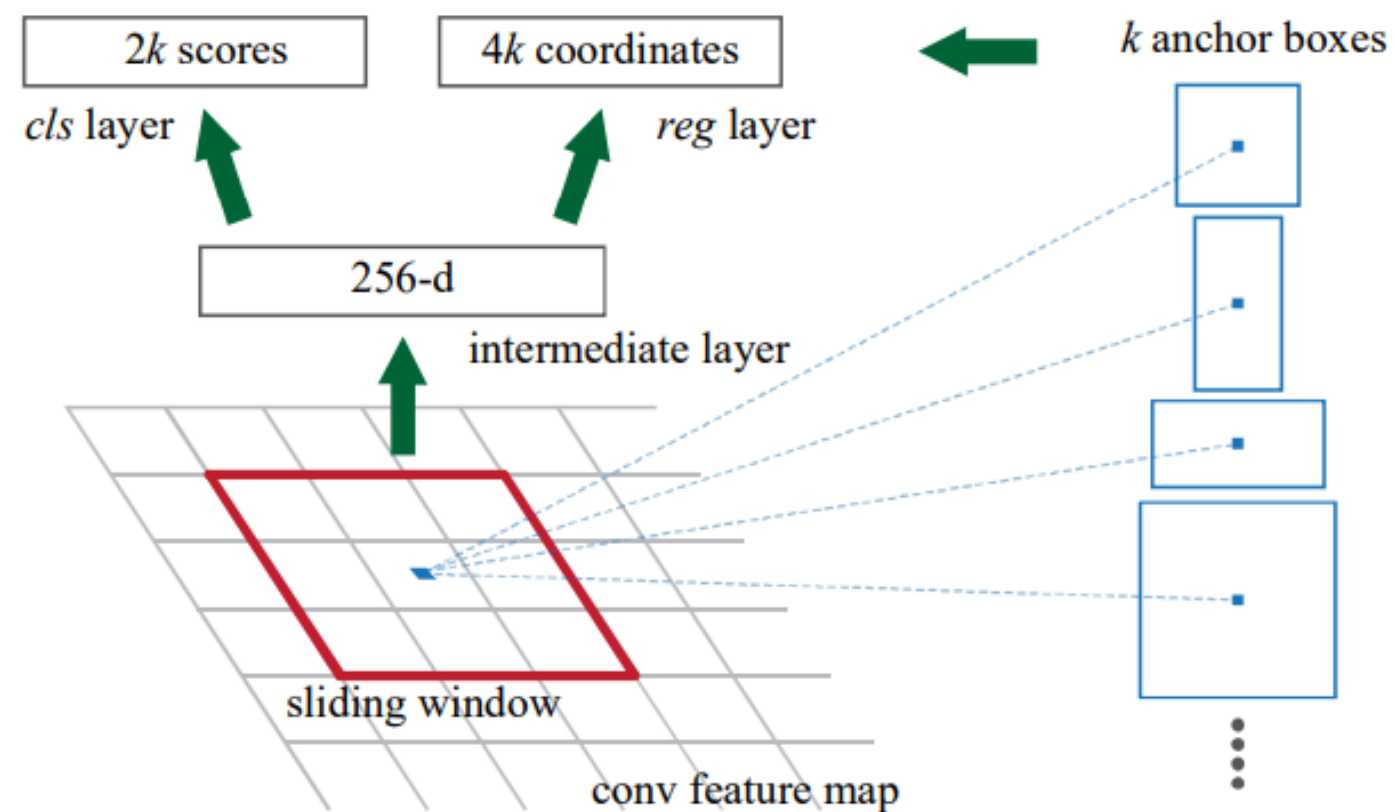
3.1 Region Proposal Networks

- 다양한 사이즈의 이미지를 입력 값으로 object score와 object proposal을 출력
- Fast R-CNN과 합성곱 신경망을 공유
- Feature map의 마지막 conv 층을 작은 네트워크가 sliding하여 저차원으로 매핑
- Regression과 Classification 수행



3.1.1 Anchors

- 각 슬라이딩 윈도우에서 Bounding box의 후보로 사용되는 상자
- reg층은 4k를, cls층은 2k만큼의 출력을 갖게 된다.
- anchor는 슬라이딩 윈도우 중심에 있으며 미리 정해진 스케일과 비율에 따라 달라진다.
- 실험에서는 3개의 스케일과 3개의 비율을 사용하여 k=9개의 앵커를 사용하였다.
- $W \times H$ 크기만큼의 conv feature map에서 $W \times H \times k$ 만큼의 앵커가 있는 것이다.



3.1.1 Translation-Invariant(이동 불변성)

- RPN에서 윈도우 사이즈를 슬라이딩 하는 방식은 이동불변성을 보장 → 위치가 어디든 같은 특징을 뽑아줌.
- CNN에서 윈도우 슬라이딩을 사용하여 convolution을 하였을 때 얻는 효과와 동일
- 모델 사이즈를 줄여준다. → 연산량 절감



Multi-Scale Anchors as Regression References

- Pyramid of anchors → 앵커라는 참조 박스를 사용해 다양한 크기와 종횡비를 처리하는 방식이 비용(시간 및 계산 자원) 면에서 효율적
- 다양한 scale과 ratio를 활용한 anchor를 통해 효율적으로 계산할 수 있다.

→ 이전 모델은 직접 필터의 크기나 스케일을 조정하며 실험을 진행했어야 함. 시간이 많이 걸리고 계산 자원 많이 사용.

→ 앵커 피라미드 방식은 앵커의 하이퍼파라미터(예: 스케일, 종횡비)를 조정함으로써 다양한 크기와 비율을 효율적으로 처리할 수 있음. 굳이 직접 하나하나 실험하지 않아도 됨.

3.1.2 Loss Function

$$L(p_i, t_i) = \frac{1}{N_{cls}} \sum_i L_{cls}(p_i, p_i^*) + \lambda \frac{1}{N_{reg}} \sum_i p_i^* L_{reg}(t_i, t_i^*)$$

- i : 미니배치에서 앵커(anchor)의 인덱스
- p_i : 모델이 예측한 "이 앵커가 객체일 확률" (0~1 사이 값)
- p_i^* : 실제 정답 (객체가 있으면 1, 없으면 0)
- t_i : 모델이 예측한 바운딩 박스 좌표 (4개 값: 중심 좌표 + 너비 + 높이)
- t_i^* : 실제 ground truth 바운딩 박스 좌표
- $L_{cls}(p_i, p_i^*)$: 이진 분류 손실 (classification loss)
- $L_{reg}(t_i, t_i^*)$: 바운딩 박스 회귀 손실 (regression loss)
- $p_i^* L_{reg}$: 회귀 손실은 양성(positive) 앵커일 때만 적용 (즉, $p_i^* = 1$ 일 때만)
- N_{cls} : 분류 손실을 정규화하는 값 (미니배치 크기, 보통 256)
- N_{reg} : 회귀 손실을 정규화하는 값 (보통 앵커 개수, 약 2400)
- λ : 분류 손실과 회귀 손실 간의 균형을 맞추는 가중치 (보통 10)

[3.1.2 Loss Function]

1) 분류 손실

$$L(p_i, t_i) = \frac{1}{N_{cls}} \sum_i L_{cls}(p_i, p_i^*) + \lambda \frac{1}{N_{reg}} \sum_i p_i^* L_{reg}(t_i, t_i^*)$$

- 객체가 있는지 없는지 예측하는 손실입니다.
- 이진 교차 엔트로피를 사용하여 두 클래스(객체/배경)을 구분합니다.
- 실제 정답 p_i^* 과 예측 확률 p_i 간의 차이를 줄이는 것이 목표입니다.

2) 회귀 손실

$$L(p_i, t_i) = \frac{1}{N_{cls}} \sum_i L_{cls}(p_i, p_i^*) + \lambda \frac{1}{N_{reg}} \sum_i p_i^* L_{reg}(t_i, t_i^*)$$

- 앵커 박스를 실제 객체 박스에 가깝게 만드는 손실입니다.
- Smooth L1 Loss를 사용하여 예측한 바운딩 박스 t_i 가 실제 값 t_i^* 와 가까워지도록 합니다.
- 오직 positive(객체포함) 앵커에서만 적용됩니다.($p_i^*=1$ 인 경우)

[3.1.3 Training RPNs]

1. 훈련 방식

- 엔드투엔드(End-to-End) 훈련
- 역전파(Backpropagation) + 확률적 경사 하강법(SGD)

2. 샘플링 전략

- "이미지 중심(Image-Centric)" 샘플링
- 256개 앵커 무작위 샘플링(전체 앵커를 학습하면 음성이 너무 많아짐)
- 양성(Positive) : 음성(Negative) 비율 최대 1:1
- 양성 샘플이 128개보다 적으면 음성 샘플로 패딩

3. 초기화 및 사전 학습

- 새로운 계층 → 평균 0, 표준편차 0.01인 가우시안 분포에서 초기화
- 합성곱 계층 → ImageNet 사전 학습 가중치 사용

[3.1.3 Training RPNs]

4. 학습률 및 하이퍼파라미터

- ZFNet → 모든 계층 튜닝
- VGGNet → conv3_1 이후 계층만 튜닝

학습률 조정

- 0.001 (60,000 미니배치)
- 0.0001 (20,000 미니배치)
- 모멘텀: 0.9 / 가중치 감소: 0.0005

5. 구현

- Caffe 프레임워크 사용

[3.2 Sharing Features for RPN and Fast R-CNN]

✗ RPN과 Fast R-CNN을 따로 훈련하면?
각각 다른 방식으로 합성곱 계층을 수정 → 비효율적!



- 💡 해결 방법: 합성곱 계층 공유!
- ✓ RPN + Fast R-CNN → 하나의 네트워크로 합성곱 계층 공유
 - ✓ 네트워크를 통합해 효율적이고 최적화된 학습 가능

[3.2 Sharing Features for RPN and Fast R-CNN]

RPN과 Fast R-CNN을 함께 학습하는 3가지 방법

1. Alternating Training

- RPN을 먼저 학습 → 생성된 제안 영역(proposals)로 Fast R-CNN 학습
- Fast R-CNN으로 조정된 네트워크를 다시 RPN 초기값으로 사용
- 이 과정을 반복(Iteration)하면서 점진적으로 최적화

채택!

2. Approximate Joint Training

- RPN과 Fast R-CNN을 하나의 네트워크로 결합하여 학습
- SGD 반복(iteration) 때마다 영역 제안 생성 후, Fast R-CNN 학습에 사용
- RPN과 Fast R-CNN 손실(loss)을 결합하여 합성곱 계층 업데이트
- 제안된 박스 좌표의 그래디언트 계산을 무시 → 완전한 학습(backpropagation)이 아님

3. Non-Approximate Joint Training

- 근사 공동 훈련의 문제점(박스 좌표 그래디언트 무시)을 해결
- RoI 풀링 계층이 박스 좌표에 대해 미분 가능해야 함
- ! 하지만 이 문제는 해결이 어려우며, RoI 워핑(RoI warping) 계층을 사용해야 함

[3.2 Sharing Features for RPN and Fast R-CNN]

4-Step Alternating Training → RPN과 Fast R-CNN이 공유된 특징(feature)을 학습하도록 최적화하는 과정

1단계: RPN 학습

- ImageNet 사전 훈련 모델을 초기값으로 사용
- region proposal task를 위해 end-to-end 방식으로 학습

2단계: Fast R-CNN 학습

- 1단계에서 생성된 영역 제안(proposals)을 사용하여 Fast R-CNN 학습 (초기화는 ImageNet 학습 모델로)
- 아직 RPN과 Fast R-CNN이 합성곱 계층을 공유하지 않음!

3단계: RPN fine-tuning

- 공통된 합성곱 계층을 고정(fix) → RPN의 나머지 계층만 미세 조정 및 학습
- RPN과 Fast R-CNN이 합성곱 계층을 공유

4단계: Fast R-CNN fine-tuning

- 공유된 합성곱 계층을 고정시키고 Fast R-CNN 미세조정 및 학습

3.3 Implementation Details

입력 이미지 조정

- 이미지의 짧은 변이 600 pixel이 되도록 크기 조정
- RPN과 Fast R-CNN 모두 single scale의 이미지로 학습 및 테스트
 - multi scale을 사용하면 속도와 정확도 사이의 균형이 좋지 않음

Anchor 설정

- 총 9개의 anchor box 사용 - 3개의 scale(128^2 , 256^2 , 512^2), 3개의 aspect ratios (1:1, 1:2, 2:1)
- 훈련할 때, 이미지 경계를 넘어서는 anchor box는 무시하여 손실 계산에 포함되지 않도록 함
- 테스트 시에는 경계를 넘는 proposal가 생성될 수 있으며, 이는 이미지 경계에 맞춰 잘라냄

Non-Maximum Suppression(NMS) 적용

- proposal는 서로 겹칠 수 있기 때문에, 중복을 줄이기 위해 classification score를 기준으로 NMS를 적용
- IoU threshold은 0.7로 고정
- NMS 후에는 class score가 높은 상위 N개의 proposal만을 사용하여 detection을 수행

04 EXPERIMENTS

[4.1 Experiments on PASCAL VOC]

(1) 다양한 region proposal 방법 비교

- **PASCAL VOC 2007 객체 탐지 벤치마크**
 - 학습 데이터 : 5000장
 - 테스트 데이터 : 5000장
 - 총 20개의 category
- **백본 모델 : ZF Net: Conv layer 5개 + FC layer 3개**

train-time region proposals		test-time region proposals		mAP (%)
method	# boxes	method	# proposals	
SS	2000	SS	2000	58.7
EB	2000	EB	2000	58.6
RPN+ZF, shared	2000	RPN+ZF, shared	300	59.9

- RPN+ZF에서 훨씬 적은 수의 Proposal를 가짐에도 불구하고 Selective Search나 EdgeBoxes보다 높은 mAP 값을 가짐

[4.1 Experiments on PASCAL VOC]

(2) Ablation Experiments on RPN

train-time region proposals		test-time region proposals		mAP (%)
method	# boxes	method	# proposals	
SS	2000	SS	2000	58.7
EB	2000	EB	2000	58.6
① RPN+ZF, shared	2000	RPN+ZF, shared	300	59.9
ablation experiments follow below				
② RPN+ZF, unshared	2000	RPN+ZF, unshared	300	58.7
SS	2000	RPN+ZF	100	55.1
③ SS	2000	RPN+ZF	300	56.8
SS	2000	RPN+ZF	1000	56.3
SS	2000	RPN+ZF (no NMS)	6000	55.2
SS	2000	RPN+ZF (no cls)	100	44.6
④ SS	2000	RPN+ZF (no cls)	300	51.4
SS	2000	RPN+ZF (no cls)	1000	55.8
⑤ SS	2000	RPN+ZF (no reg)	300	52.1
SS	2000	RPN+ZF (no reg)	1000	51.3
⑥ SS	2000	RPN+VGG	300	59.2

① ② Convolution layer 공유 여부

RPN과 Fast R-CNN의 conv layer를 공유하지 않고 따로 학습하면 정확도가 58.7%로 다소 낮아짐

③ Selective Search로 학습하고 RPN을 사용한 경우

학습과 테스트의 proposal 방식이 다르기 때문에 성능이 낮아짐

④ RPN의 cls layer 제거한 경우

proposal가 300개인 경우, 성능이 51.4%로 저하됨
→ cls score가 좋은 proposal을 고르는 데 핵심적임

⑤ RPN의 reg layer 제거한 경우

proposal가 300개인 경우, 성능이 52.1%로 저하됨
→ high-quality proposal은 reg layer에 크게 의존

⑥ VGG-16을 사용한 경우

VGG 기반 RPN proposal를 넣었을 때 mAP가 59.2%로 성능이 향상함

04 EXPERIMENTS

[4.1 Experiments on PASCAL VOC]

(3) Performance of VGG-16

[Table3] VGG-16을 사용한 경우 성능 비교

method	# proposals	data	mAP (%)
SS	2000	07	66.9 [†]
SS	2000	07+12	70.0
RPN+VGG, unshared	300	07	68.5
RPN+VGG, shared	300	07	69.9
RPN+VGG, shared	300	07+12	73.2
RPN+VGG, shared	300	COCO+07+12	78.8

- SS보다 RPN+VGG unshared의 성능이 약간 더 높음
- Feature sharing을 적용 / 07+12 데이터를 사용한 경우 mAP 69.9%로 SS보다 훨씬 높은 성능을 가진다

[Table5] 속도 비교

model	system	conv	proposal	region-wise	total	rate
VGG	SS + Fast R-CNN	146	1510	174	1830	0.5 fps
VGG	RPN + Fast R-CNN	141	10	47	198	5 fps
ZF	RPN + Fast R-CNN	31	3	25	59	17 fps

*fps : 초당 처리 이미지의 수

- selective search보다 RPN이 훨씬 빠른 속도를 보임 (total 1830 → 198)
- RPN에서 proposal 처리 시간이 10ms로 짧아짐

[Table4] PASCAL VOC 2012 test set에 대한 결과

method	# proposals	data	mAP (%)
SS	2000	12	65.7
SS	2000	07++12	68.4
RPN+VGG, shared [†]	300	12	67.0
RPN+VGG, shared [‡]	300	07++12	70.4
RPN+VGG, shared [§]	300	COCO+07++12	75.9

- VOC 2007 trainval+test와 VOC 2012 trainval을 합쳐 학습한 경우 mAP가 70.4%를 기록
- COCO dataset까지 포함한 경우 75.9%로 가장 높은 성능을 보임

[4.1 Experiments on PASCAL VOC]

(4) Sensitivities to Hyper-parameters

[Table8] Anchor 설정

settings	anchor scales	aspect ratios	mAP (%)
1 scale, 1 ratio	128^2	1:1	65.8
	256^2	1:1	66.7
1 scale, 3 ratios	128^2	{2:1, 1:1, 1:2}	68.8
	256^2	{2:1, 1:1, 1:2}	67.9
3 scales, 1 ratio	{ $128^2, 256^2, 512^2$ }	1:1	69.8
3 scales, 3 ratios	{ $128^2, 256^2, 512^2$ }	{2:1, 1:1, 1:2}	69.9

- 각 위치에 하나의 앵커만 사용하면 mAP가 가장 낮음
- 논문의 방식대로 3가지 scales, 3가지 aspect ratios를 사용하는 것이 성능이 가장 좋음

=> 이 결과는 다양한 크기의 앵커를 사용하는 것이 좋은 성능으로 이어진다는 걸 보여줌

[Table9] Loss의 λ parameter 값에 따른 성능 비교

λ	0.1	1	10	100
mAP (%)	67.2	68.9	69.9	69.1

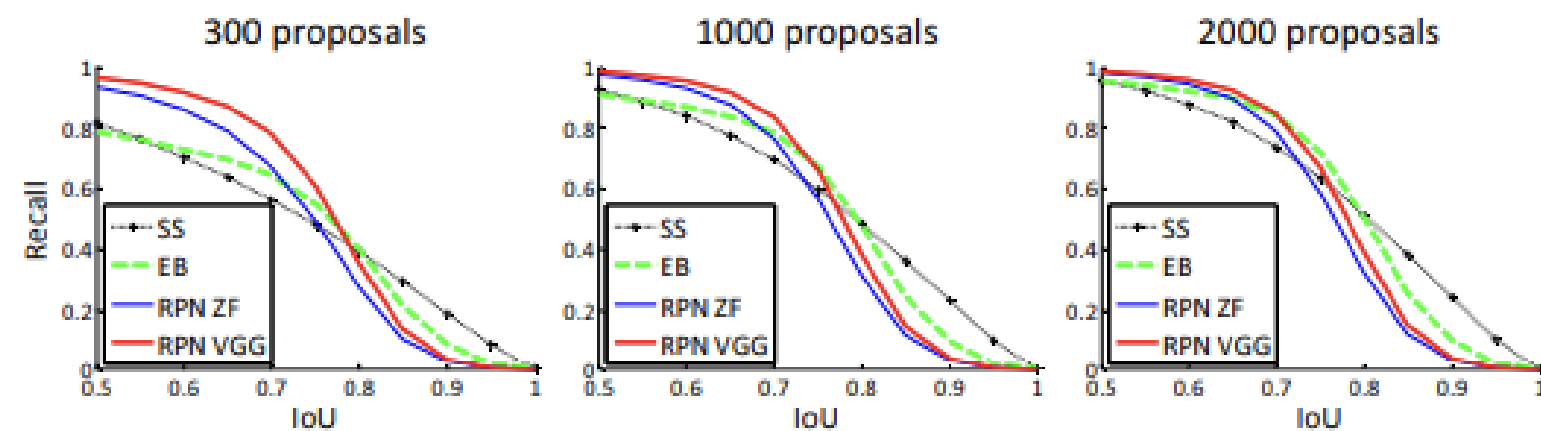
- 기본적으로 우리는 $\lambda = 10$ 을 사용하는데, 이는 두 손실항을 거의 비슷한 비중으로 반영함
- 실험 결과, λ 값을 1부터 100까지 변화시켜도 mAP는 약 1%만 변화

=> 이는 λ 값에 대해 결과가 매우 민감하지 않다는 것을 보여줌

04 EXPERIMENTS

[4.1 Experiments on PASCAL VOC]

(5) Analysis of Recall-to-IoU



[Figure4] 다양한 IoU 기준에서 proposal recall을 분석

- proposal의 수가 2000개에서 300개로 줄어들어도 큰 성능의 차이가 없음
- RPN이 적은 Proposal을 통해서도 물체를 잘 찾아낼 수 있음을 의미

(6) One-Stage Detection vs. Two-Stage Proposal + Detection

	proposals		detector	mAP (%)
Two-Stage	RPN + ZF, unshared	300	Fast R-CNN + ZF, 1 scale	58.7
One-Stage	dense, 3 scales, 3 aspect ratios	20000	Fast R-CNN + ZF, 1 scale	53.8
One-Stage	dense, 3 scales, 3 aspect ratios	20000	Fast R-CNN + ZF, 5 scales	53.9

- ZF 모델을 사용한 one-stage System이 2-stage보다 낮은 성능을 보임
- 이를 통해 2-stage가 성능이 더 좋은 것을 확인할 수 있음

04 EXPERIMENTS

4.2 Experiments on MS COCO

MS COCO dataset

- 학습용 이미지 80,000장 / 검증용 이미지 40,000장
테스트용 이미지 20,000장
- Anchor box의 scale을 4개로 확장
→ 이는 작은 객체를 처리하기 위한 조치

[Table11] MS COCO dataset에서 성능평가(VGG-16)

method	proposals	training data	COCO val		COCO test-dev	
			mAP@.5	mAP@ [.5, .95]	mAP@.5	mAP@ [.5, .95]
Fast R-CNN [2]	SS, 2000	COCO train	-	-	35.9	19.7
Fast R-CNN [impl. in this paper]	SS, 2000	COCO train	38.6	18.9	39.3	19.3
Faster R-CNN	RPN, 300	COCO train	41.5	21.2	42.1	21.5
Faster R-CNN	RPN, 300	COCO trainval	-	-	42.7	21.9

- Faster R-CNN은 단 300개의 Proposal만 사용했음에도 Fast R-CNN보다 mAP@0.5 / mAP@[.5, .95]가 더 높음
- 학습 데이터를 더 많이 쓰면 성능이 더 좋아짐

4.3 From MS COCO to PASCAL VOC

COCO에서 학습한 모델을 PASCAL VOC에 평가

training data	2007 test	2012 test
VOC07	69.9	67.0
VOC07+12	73.2	-
VOC07++12	-	70.4
COCO (no VOC)	76.1	73.0
COCO+VOC07+12	78.8	-
COCO+VOC07++12	-	75.9

- 1단계 : COCO 데이터셋으로 학습 시킨 모델을 PASCAL VOC에 평가
 - mAP 76.1% → VOC07+12로 학습한 모델(73.2%)보다 높음
- 2단계 : COCO에서 학습한 모델을 VOC 데이터로 파인튜닝
 - PASCAL VOC 2007 테스트 세트에서 mAP 78.8%

5. Conclusion

- 효율적이고 빠르며 정확한 RPN 제안
- RPN과 Fast R-CNN이 convolution feature를 공유함으로써, region proposal 단계의 계산 비용을 줄임
- 이 구조 덕분에 실시간에 가까운 속도로 객체 탐지가 가능해짐
- 학습된 RPN은 Region Proposal의 품질을 향상시킴

감사합니다

[Faster R-CNN 논문 리뷰]

24기 분석 조희선, 서혜현